

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 吴竞宸 学号 2024K8009929022 专业 计算机科学与技术
实验项目编号 3 实验名称 定制功能型处理器设计

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 `/home/serve-ide/cod-lab/reports` 目录下(注意: reports 全部小写)。文件命名规则: `prjN.pdf`, 其中 `prj` 和后缀名 `pdf` 为小写, `N` 为 1 至 4 的阿拉伯数字。例如: `prj1.pdf`。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: `prj5-projectname.pdf`, 其中“-”为英文标点符号的短横线。文件命名举例: `prj5-dma.pdf`。具体要求详见实验项目 5 讲义。
- 注 2: 使用 `git add` 及 `git commit` 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 `git push` 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

完整实验流程包括设计、开发、调试三个步骤:

- 设计步骤为在代码编写前的准备阶段, 主要包括模块划分和模块结构设计, 并绘制设计图纸。
- 开发步骤为在 S-IDE 集成开发环境中进行 Verilog/C 代码编写, 并实现完整功能。
- 调试步骤包括本地仿真验证后推送至云平台进行 FPGA 原型验证, 以及后续迭代调试的全过程。

一、设计成果(对设计流程的总结。需附上设计结构图, 可用 PPT 或者在线绘图软件绘制逻辑结构框图后保存为 PDF, 再插入到 L^AT_EX 中)

1 增加真实内存的 RISC-V 处理器设计

将访存的逻辑从 prj2 中的理想内存修改为本实验所要求的真实内存, 主要改造了访存接口、状态机逻辑。

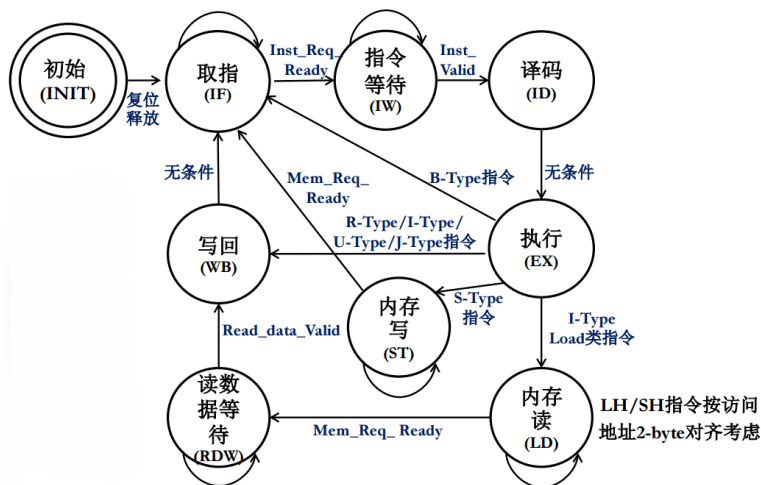


图 1: 状态转移图

2 外设控制器设计

修改了 `printf.c`, 实现向 UART 控制器传送字符串的程序功能。

3 性能计数器

实现了周期计数器。

二、 开发成果(对代码开发流程的总结。需附上包含注释的 Verilog HDL 关键代码段、相应信号的仿真波形和信号变化的说明)

1 增加真实内存的 RISC-V 处理器设计

主要的修改如下：

1. 更新了状态机的逻辑,PC、取指、访存等逻辑也做了对应修改。

```
// States Transference
always @(*)
begin
    case(current_state)
        INIT:
            next_state=IF;
        IF:
            if(Inst_Req_Ready)
                next_state=IW;
            else
                next_state=IF;
        IW:
            if(Inst_Valid)
                next_state=ID;
            else
                next_state=IW;
        ID:
            next_state=EX;
        EX:
            case(opcode)
                `OPCODE_BRANCH: next_state=IF;
                `OPCODE_JAL, `OPCODE_JALR, `OPCODE_LUI, `OPCODE_AUIPC, `OPCODE_CALC, `OPCODE_IMM:
                    next_state=WB;
                `OPCODE_LOAD: next_state=LD;
                `OPCODE_STORE: next_state=ST;
                default: next_state=INIT;
            endcase
        ST:
            if(Mem_Req_Ready)
                next_state=IF;
            else
                next_state=ST;
        LD:
            if(Mem_Req_Ready)
                next_state=RDW;
            else
                next_state=LD;
```

```

RDW:
    if(Read_data_Valid)
        next_state=WB;
    else
        next_state=RDW;
WB:
    next_state=IF;
default:
    next_state=INIT;
endcase
end

```

2. 用组合逻辑控制握手信号和传回数据。

```

// Handshake Signal
assign Inst_Req_Valid  = is_IF | 1'b0;
assign Inst_Ready  = is_INIT | is_IW | 1'b0;
assign Read_data_Ready  = is_RDW | is_INIT | 1'b0;
assign MemRead    = is_LD | 1'b0;
assign MemWrite   = is_ST | 1'b0;
assign inst_retire = {RF_wen, RF_waddr, RF_wdata, cur_pc};

```

其余部分与 prj2 中已经实现的多周期处理器大致相同。

2 外设控制器设计

puts 函数如下所示:

```

int
puts(const char *s)
{
    int i=0;
    while(s[i] != '\0')
    {
        while(*(uart+UART_STATUS/sizeof(unsigned int)) & UART_TX_FIFO_FULL)
            continue; // FIFO is full
        *(uart+UART_TX_FIFO/sizeof(unsigned int))=s[i++];
    }
    return i;
}

```

这里实现了 STAT_REG 寄存器的第 3 比特为 1 表示发送队列到达满状态,暂停写操作的逻辑。

3 性能计数器

custom_cpu.v 中新增了周期计数器。

```

// Performance Counter
reg [31:0] cycle_cnt;
assign cpu_perf_cnt_0 = cycle_cnt;
always @ (posedge clk)

```

```
begin
    if (rst == 1'b1)
        cycle_cnt <= 32'd0;
    else
        cycle_cnt <= cycle_cnt + 32'd1;
    end
end
```

三、 学习总结(总结设计、开发与调试过程中遇到的问题, 及实验中的收获与不足)

1 问题总结(实验过程中遇到的问题、对问题的思考过程及解决方法)

序号	问题类别	出错模块	问题现象	产生原因	解决办法
1	行为仿真	UART	无法得到输出	框架有问题	直接推送
2	行为仿真	CPU	microbench 没有输出	等的不够久	耐心等待

表 1: 实验问题总结

2 收获总结(在数字电路设计方法、RTL 编写方法、数字电路验证方法及其他方面的收获)

类别	名称	体会	后续计划
亮点	分块设计	便于维护	继续使用
不足	RTL 编写不熟练	很难下手	努力学习

表 2: 收获总结

四、 扩展思考(对讲义中思考题(如有)的理解和回答)

• volatile 关键字的作用是什么? 如果去掉会出现什么后果?

volatile 关键字的核心作用是告诉编译器, 该变量的值可能会在编译器无法预知的时刻被外部硬件或异步事件改变。

具体到这段 UART 控制器的代码, 主要是禁止编译器优化, 强制每次都进行真实的内存访问。

如果去掉了 volatile, 状态判断可能会陷入死循环。

五、 实验时长

在课后, 你花费了大约 9 小时完成此次实验, 其中各个阶段的用时如表 3 所示。

步骤	时长(小时)	说明
设计	1	无
开发	5	比较难以下手
调试	3	UART 花了很久调试

表 3: 实验时长